Student Performance Prediction Using Machine Learning

Comparing Behavioural Factors vs Acdemic History

Introduction

The project explores the factors influencing student academic performance and builds predictive models to estimate final grades (G3). The analysis compares two approaches -Model A: Includes prior academic performance (G1, G2) -Model B: Excludes prior academic performance to assess behavioral and environmental impact

Load Data

```
import pandas as pd
df = pd.read_csv('student-mat.csv')
df.head()
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1

5 rows × 33 columns

Data Overview

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
```

```
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    object
 16  famsup      395 non-null    object
 17  paid        395 non-null    object
 18  activities  395 non-null    object
 19  nursery     395 non-null    object
 20  higher      395 non-null    object
 21  internet    395 non-null    object
 22  romantic    395 non-null    object
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
```

Data Cleaning

```
df.isnull().sum()
```

	0
school	0
sex	0
age	0
address	0
famsize	0
Pstatus	0
Medu	0
Fedu	0
Mjob	0
Fjob	0
reason	0
traveltime	0
studytime	0
failures	0
schoolsup	0
famsup	0
paid	0

```
df.duplicated().sum()
```

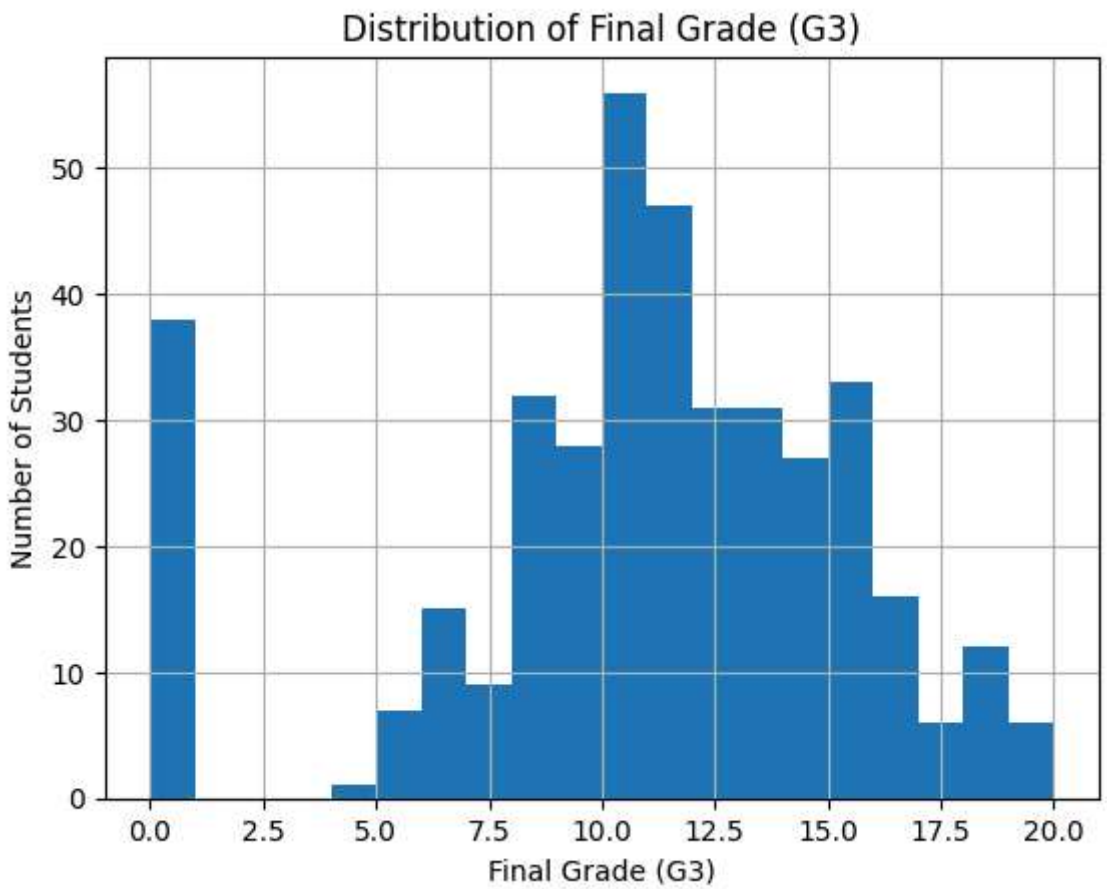
```
np.int64(0)
```

Exploratory Data Analysis

Distribution of Student Performance

```
import matplotlib.pyplot as plt
plt.figure()
df['G3'].hist(bins=20)
plt.title('Distribution of Final Grade (G3)')
plt.xlabel('Final Grade (G3)')
```

```
plt.ylabel('Number of Students')
plt.show()
```

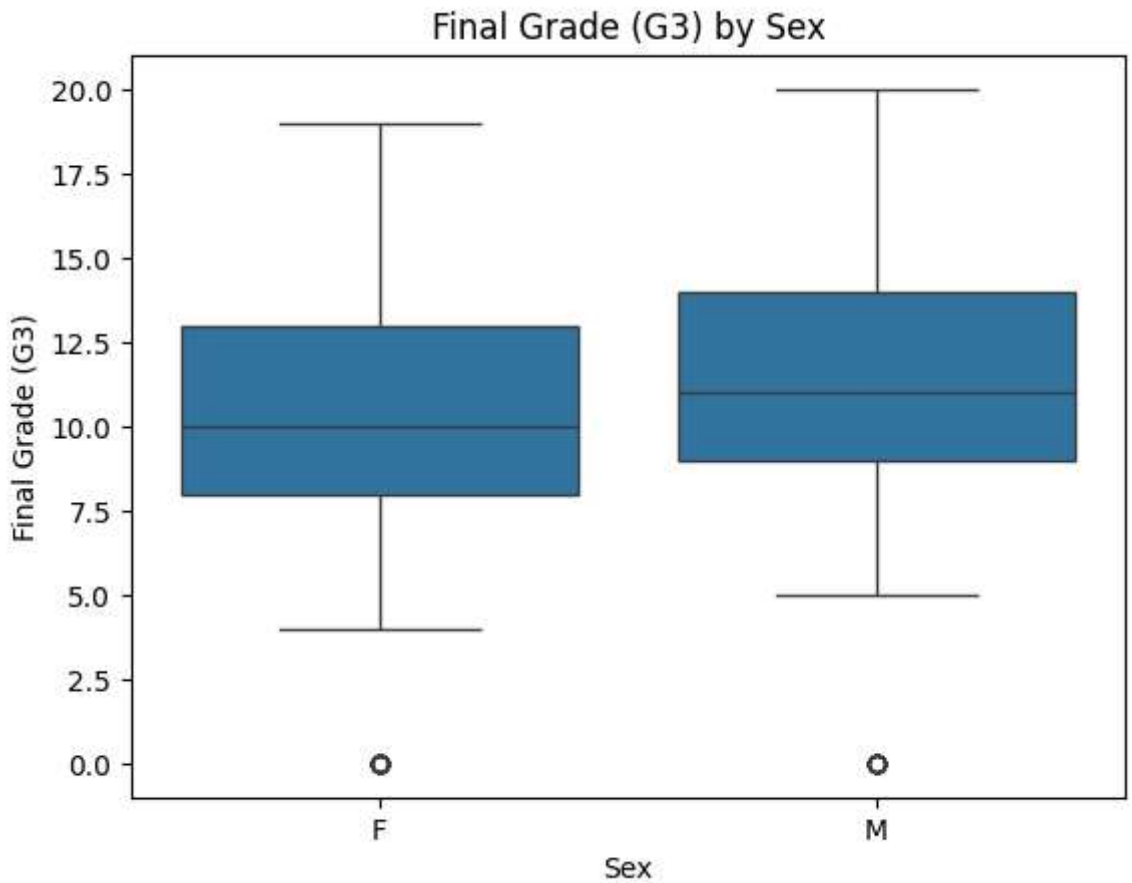


Insight

The distribution of final grades shows that most students score between mid-range values, with concentration around the average. This indicates that overall student performance is moderate with relatively few extremely high or low performers. The presence of a few low -score outliers suggests that a small group of students may be struggling significantly

Performance by Gender

```
import seaborn as sns
plt.figure()
sns.boxplot(x='sex', y='G3', data = df)
plt.title('Final Grade (G3) by Sex')
plt.xlabel('Sex')
plt.ylabel('Final Grade (G3)')
plt.show()
```

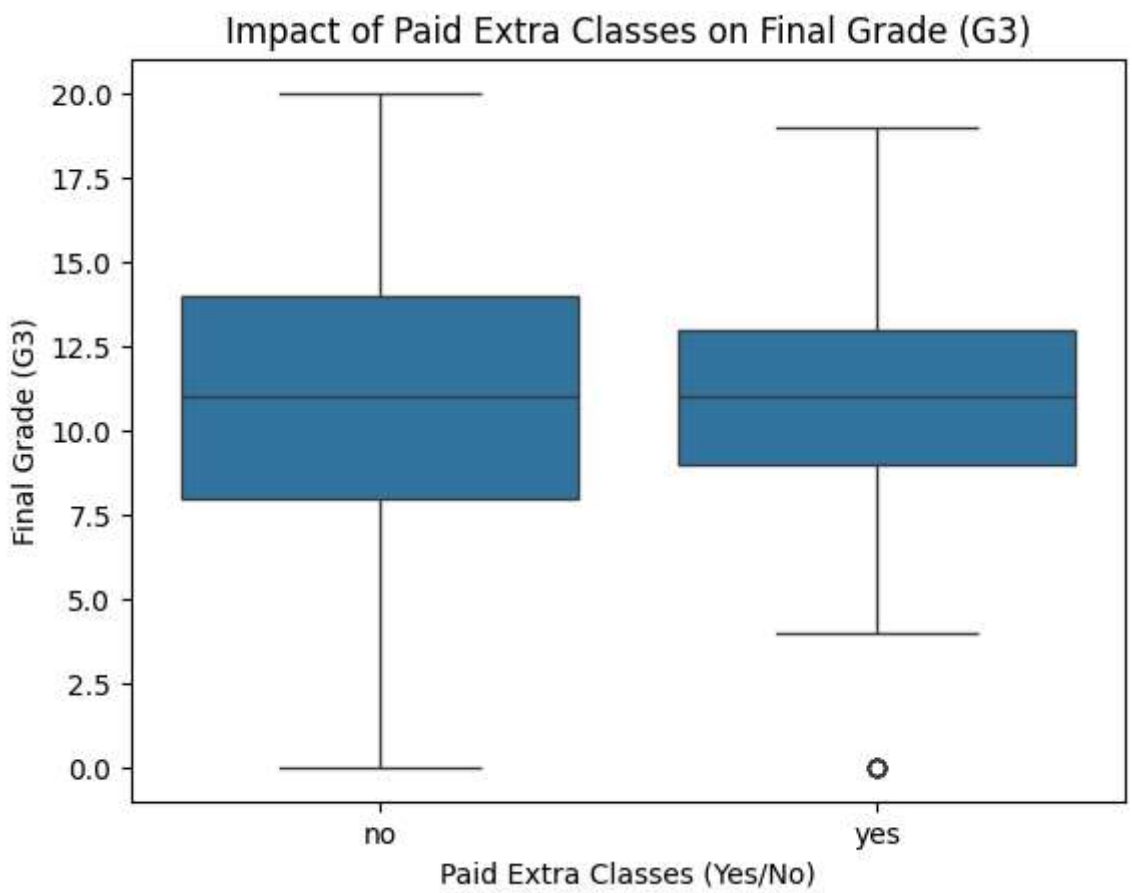


Insight

The analysis shows a slight variation in performance between genders; however, the difference is not substantial. Both groups exhibit a similar distribution of scores, suggesting that gender is not a strong determinant of academic performance compared to other influencing factors

Performance by Paid Extra Classes

```
plt.figure()
sns.boxplot(x='paid', y='G3', data = df)
plt.title('Impact of Paid Extra Classes on Final Grade (G3)')
plt.xlabel('Paid Extra Classes (Yes/No)')
plt.ylabel('Final Grade (G3)')
plt.show()
```



Insight

Students who attend paid extra classes generally demonstrate slightly higher performance compared to those who do not . This indicates that additional academic support contributes positively to student outcomes, although the difference is moderate, suggesting other facttors also influence performance

Define Target

```
y=df['G3']
```

Define Features

```
X=df.drop(['G1', 'G2', 'G3'], axis = 1)
```

Encode Categorical

```
X = pd.get_dummies(X, drop_first=True)
X.head()
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	...	guardian_mother	guardi
0	18	4	4	2	2	0	4	3	4	1	...	True	
1	17	1	1	1	2	0	5	3	3	1	...	False	
2	15	1	1	1	2	3	4	3	2	2	...	True	
3	15	4	2	1	3	0	3	2	2	1	...	True	
4	16	3	3	1	2	0	4	3	2	1	...	False	

5 rows × 39 columns

Split

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Train

```
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor()
model.fit(X_train,y_train)
```

```
▾ RandomForestRegressor
RandomForestRegressor()
```

Predict

```
y_pred = model.predict(X_test)
```

Evaluate

```
from sklearn.metrics import mean_absolute_error, r2_score
print('MAE:', mean_absolute_error(y_test,y_pred))
print('R2:', r2_score(y_test,y_pred))
```

```
MAE: 3.1070886075949367
R2: 0.2461262088581877
```

Define Target2

```
y =df['G3']
```

Define features2

```
X=df.drop(['G3'], axis=1)
```

Encode Categorical

```
X= pd.get_dummies(X,drop_first=True)
X = X.astype(int)
X.head()
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	...	guardian_mother	guardi
0	18	4	4	2	2	0	4	3	4	1	...	1	
1	17	1	1	1	2	0	5	3	3	1	...	0	
2	15	1	1	1	2	3	4	3	2	2	...	1	
3	15	4	2	1	3	0	3	2	2	1	...	1	
4	16	3	3	1	2	0	4	3	2	1	...	0	

5 rows × 41 columns

Train

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Fit

```
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor()
model.fit(X_train,y_train)
```

```
RandomForestRegressor
RandomForestRegressor()
```

Predict

```
y_pred = model.predict(X_test)
```

Evaluate

```
from sklearn.metrics import mean_absolute_error, r2_score
print('MAE:',mean_absolute_error(y_test,y_pred))
print('R2:',r2_score(y_test,y_pred))
```

```
MAE: 1.1764556962025314
R2: 0.8090088019254212
```

Features Importance

```
feature_importance = pd.Series(model.feature_importances_,index = X.columns)
feature_importance.sort_values(ascending = False).head(10)
```

	0
G2	0.787655
absences	0.108417
reason_home	0.018612
schoolsup_yes	0.008912
age	0.008796
famrel	0.006260
G1	0.006081
goout	0.005565
health	0.004314
activities_yes	0.003772

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.